DEVOPS FINAL CAPSTONE PROJECT

OpsDesk

Enterprise Cloud Incident & Site Reliability Engineering (SRE) Hub

Objective: Demonstrate an end-to-end automated DevOps lifecycle, from workstation commit to an auto-scaling, monitored cloud infrastructure.

FastAPI · Python 3.12

React 18 · Glassmorphism UI

PostgreSQL 16

Docker

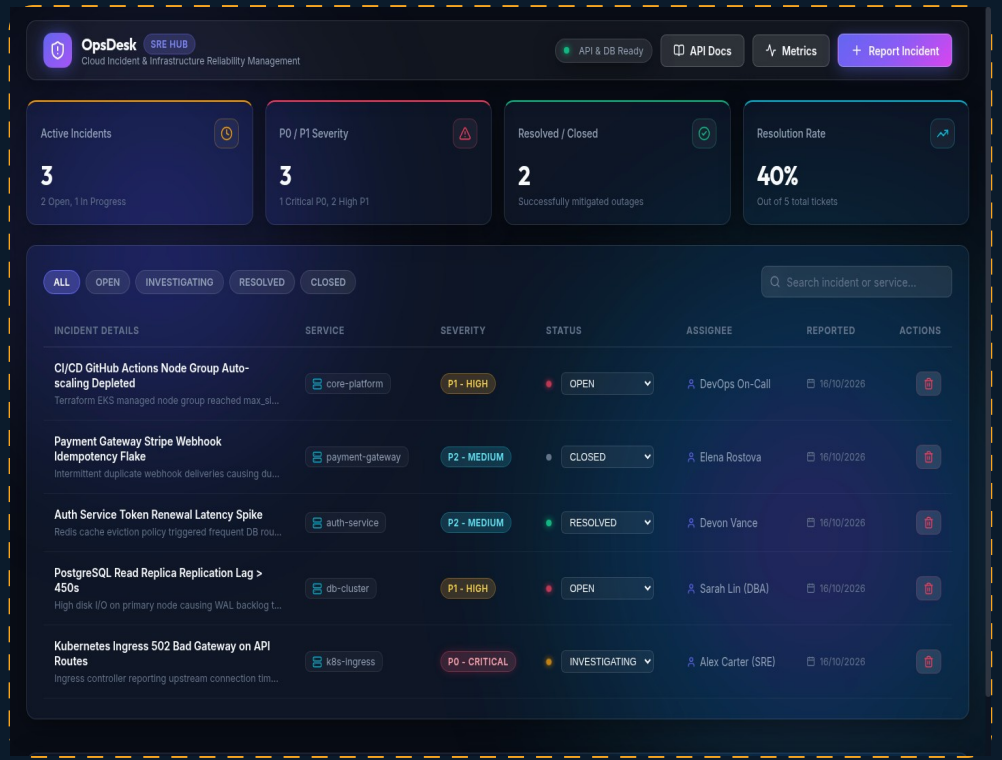
Kubernetes / Helm

AWS EKS

Terraform

Prometheus

Grafana



OpsDesk dashboard — React 18 glassmorphism UI

9 capstone modules

5 pipeline stages

17 AWS resources

ARCHITECTURE

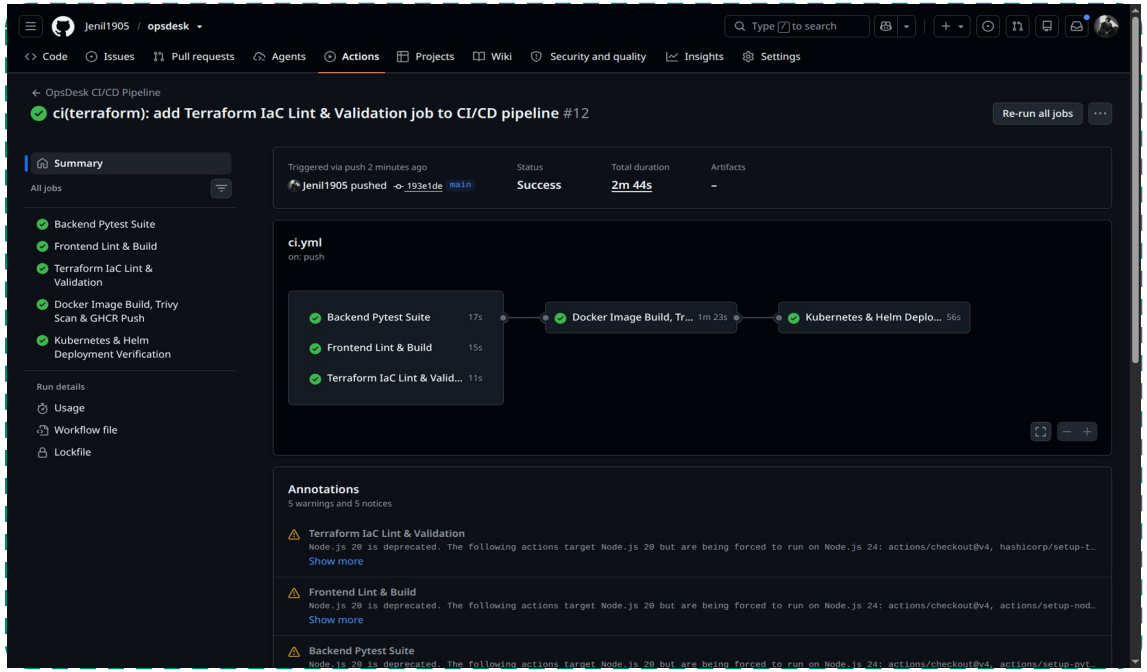
End-to-End System Architecture

1. Git Push
2. GitHub Actions CI/CD
3. Pytest & Vite Build
4. Terraform AWS IaC
5. Trivy Vulnerability Scan
6. Docker & GHCR
7. Kubernetes & Helm Deploy

Layering: Clean separation of UI, stateless REST API, persistent relational storage, and auto-scaling container workloads.

Infrastructure: Multi-AZ AWS VPC (10.0.0.0/16) powering a managed Kubernetes v1.30 cluster.

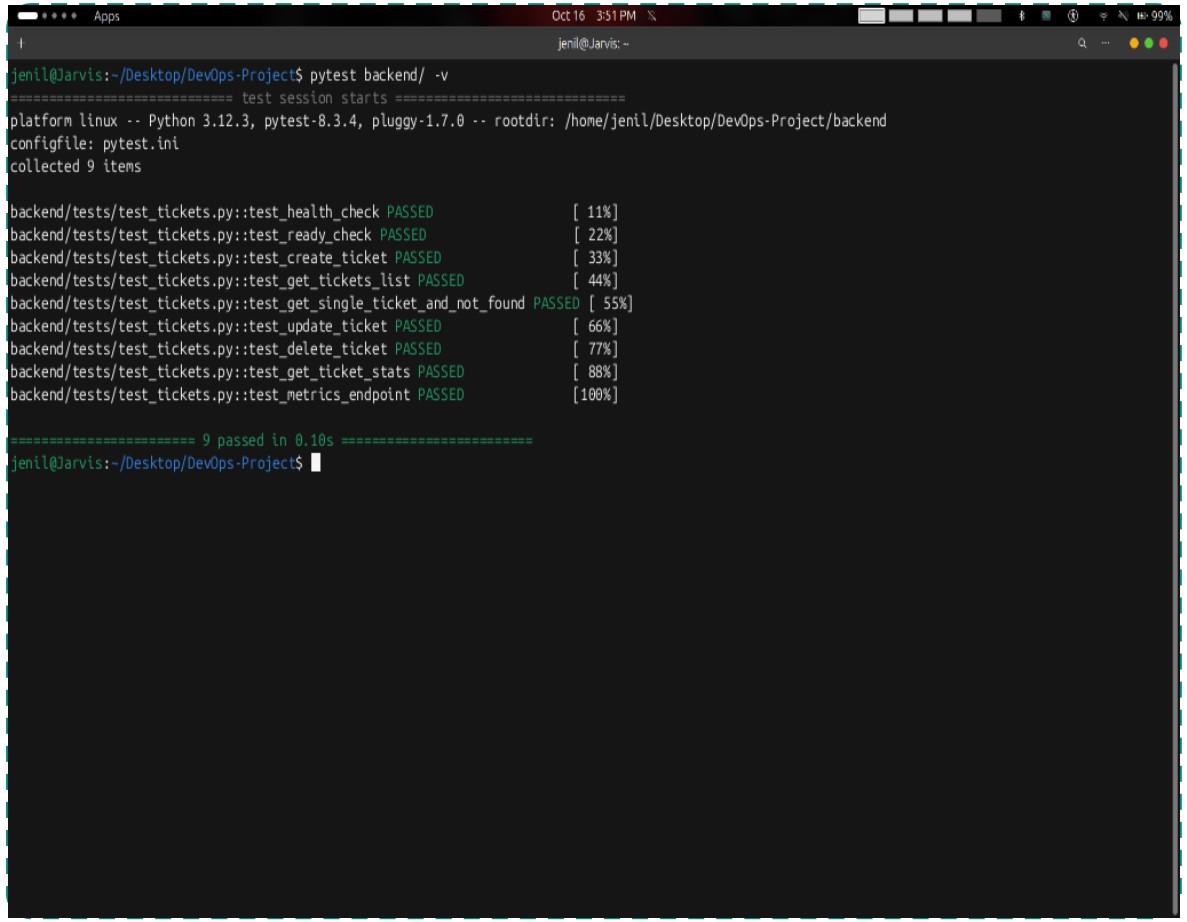
Quality gates: Tests and security scans must pass before any artifact reaches Kubernetes.



GitHub Actions — full CI/CD pipeline run

QUALITY GATE 1

Automated Testing & Code Quality



Pytest terminal — 9 passed

9/9

tests passing

<0.15s

full suite runtime

Framework: Pytest 8.3 with isolated, in-memory SQLite fixtures for hermetic runs.

Coverage: CRUD, health/readiness probes, metrics scrape, and SLA analytics.

Feedback loop: Sub-second execution gives developers near-instant results.

QUALITY GATES 2 & 3

CI/CD Pipeline & DevSecOps

Automation Engine

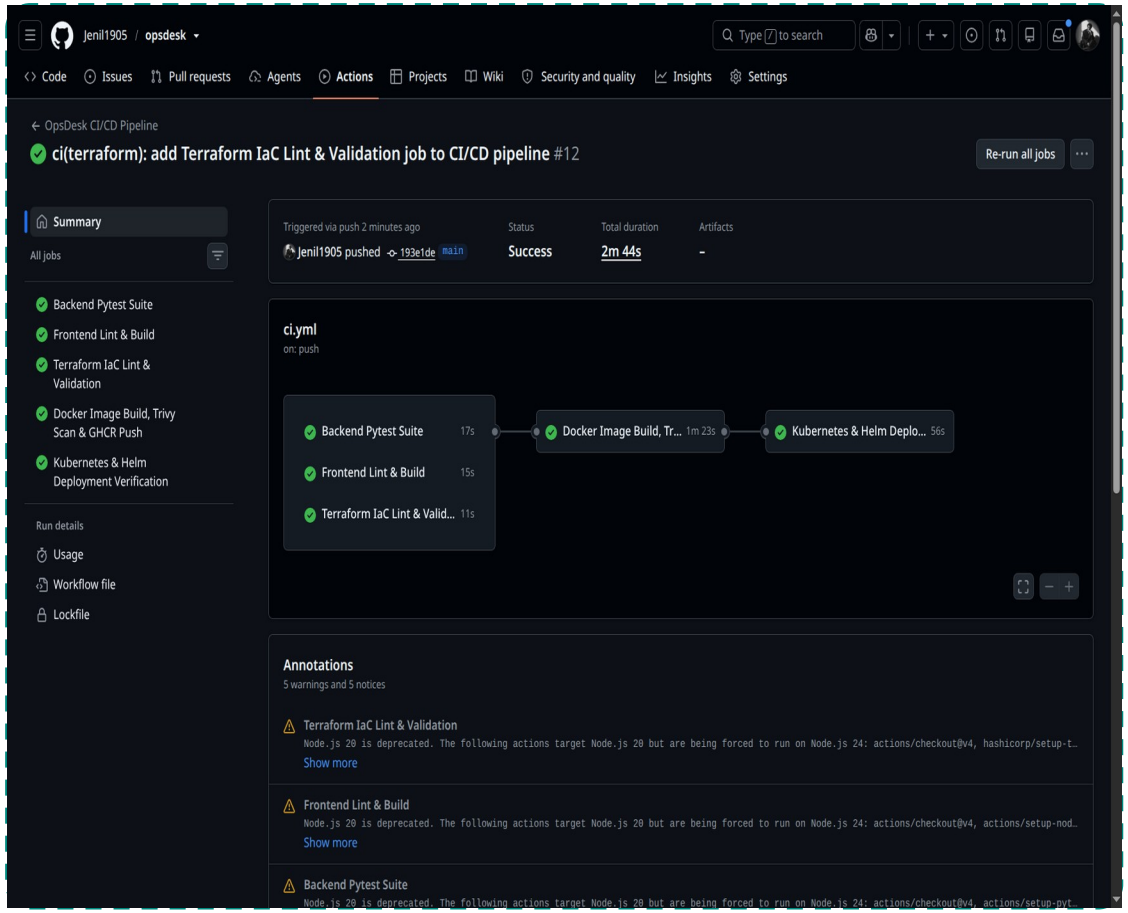
GitHub Actions workflow runs on every push to main and on pull requests.

DevSecOps Gate

Aqua Security Trivy scans backend and frontend images for CRITICAL/HIGH CVEs.

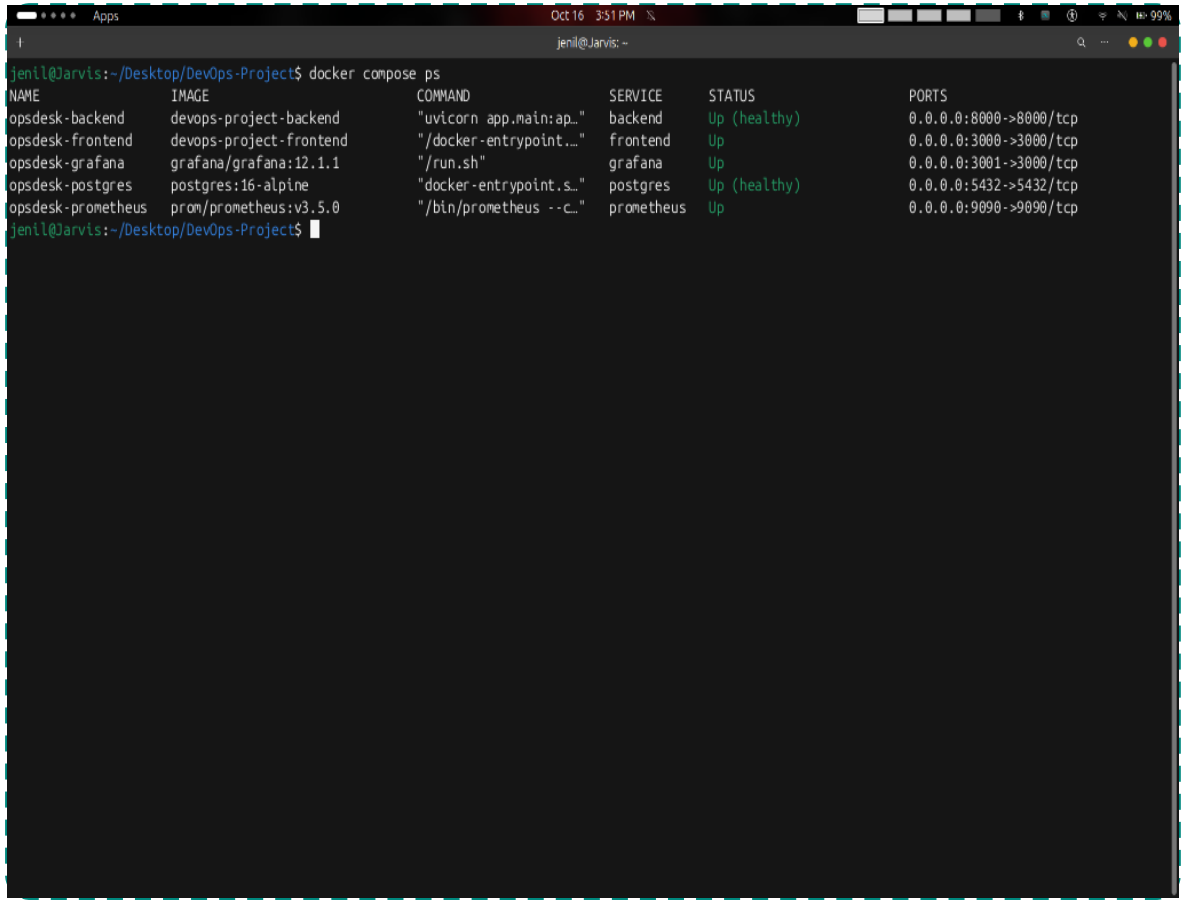
Immutable Artifacts

Docker Buildx builds non-root images, tagged with Git commit SHAs and pushed to GHCR.



All 5 pipeline stages passed

Containerization & Local Development



Docker Compose — 5 services running

Multi-service stack: 5 containers — PostgreSQL 16, FastAPI backend, React frontend, Prometheus v3, Grafana 12.

Production hardening: Non-root unprivileged user (appuser, UID 10001) for a minimal attack surface.

Environment parity: Standardized env variables keep Docker Compose and Kubernetes consistent.

```
$ docker compose up -d
```

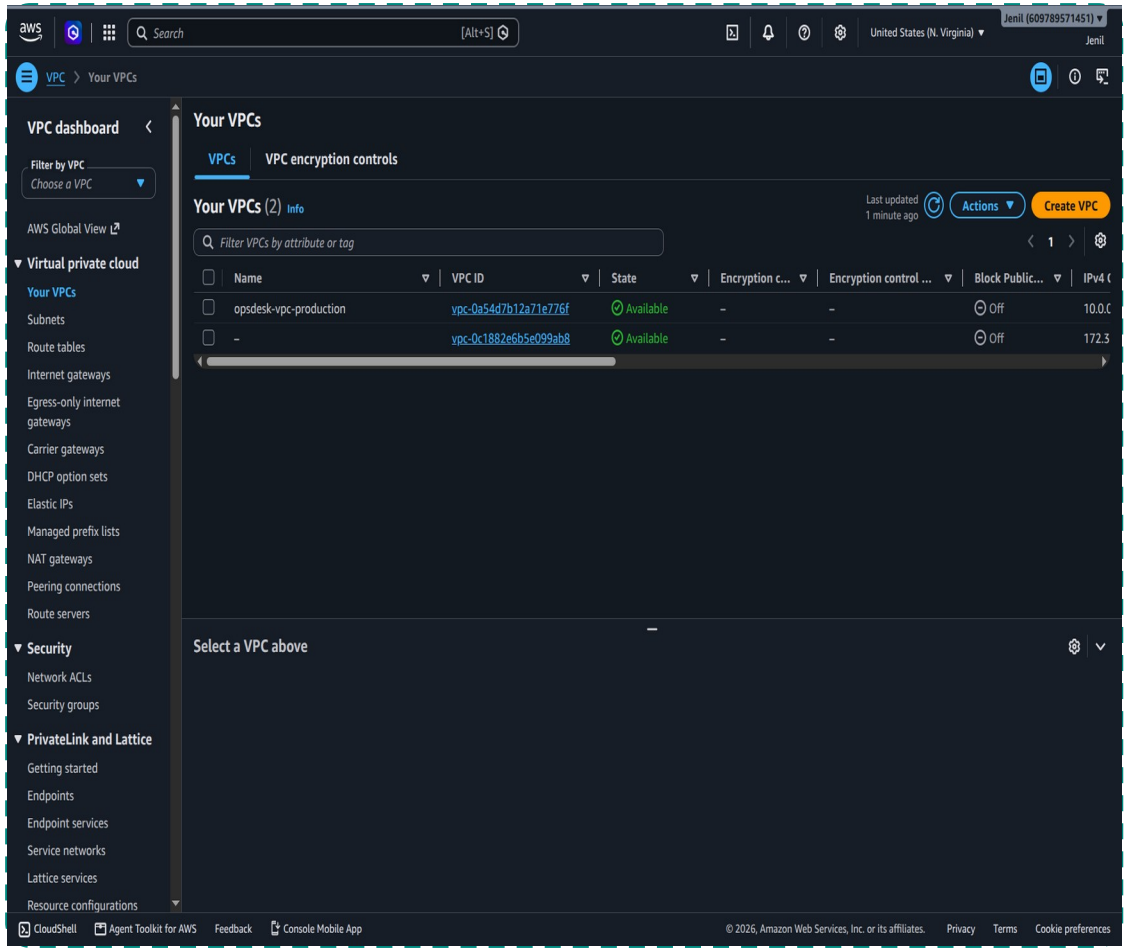
Infrastructure as Code with Terraform

IaC engine: Modular Terraform using the HashiCorp AWS Provider ~> 5.0.

Network topology: Custom VPC, 2 public subnets with an Internet Gateway for ingress, 2 private subnets for worker nodes, and route tables.

Verification: A dry-run terraform plan validated 17 cloud resources before live execution.

17	10.0.0.0/16	2 + 2
resources planned	VPC CIDR	public / private subnets



AWS Console — production VPC, us-east-1

CLOUD ORCHESTRATION

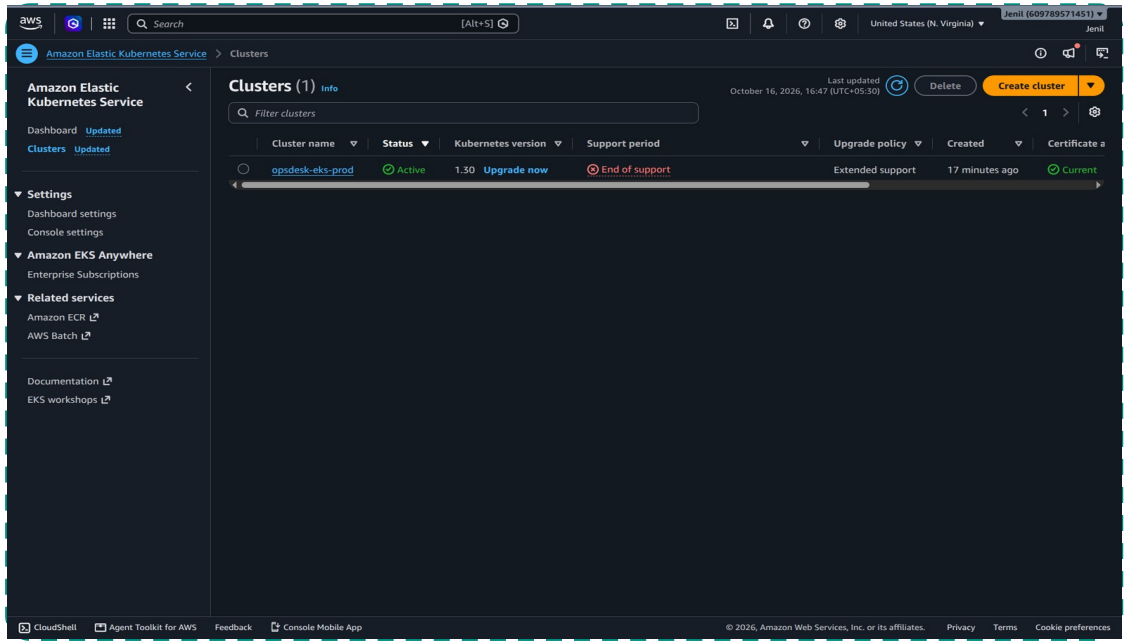
Live AWS EKS Kubernetes Cluster

v1.30	AL2023	Multi-AZ	HPA
EKS Kubernetes version	Amazon Linux 2023 nodes	managed auto-scaling workers	horizontal pod autoscaling

Cluster: AWS EKS v1.30 provisioned via Terraform (opsdesk-eks-prod).

Node group: Managed auto-scaling workers on AL2023 across multiple availability zones.

Packaging: Helm v3 chart with ConfigMaps, ClusterIP services, Ingress rules, and HPAs.



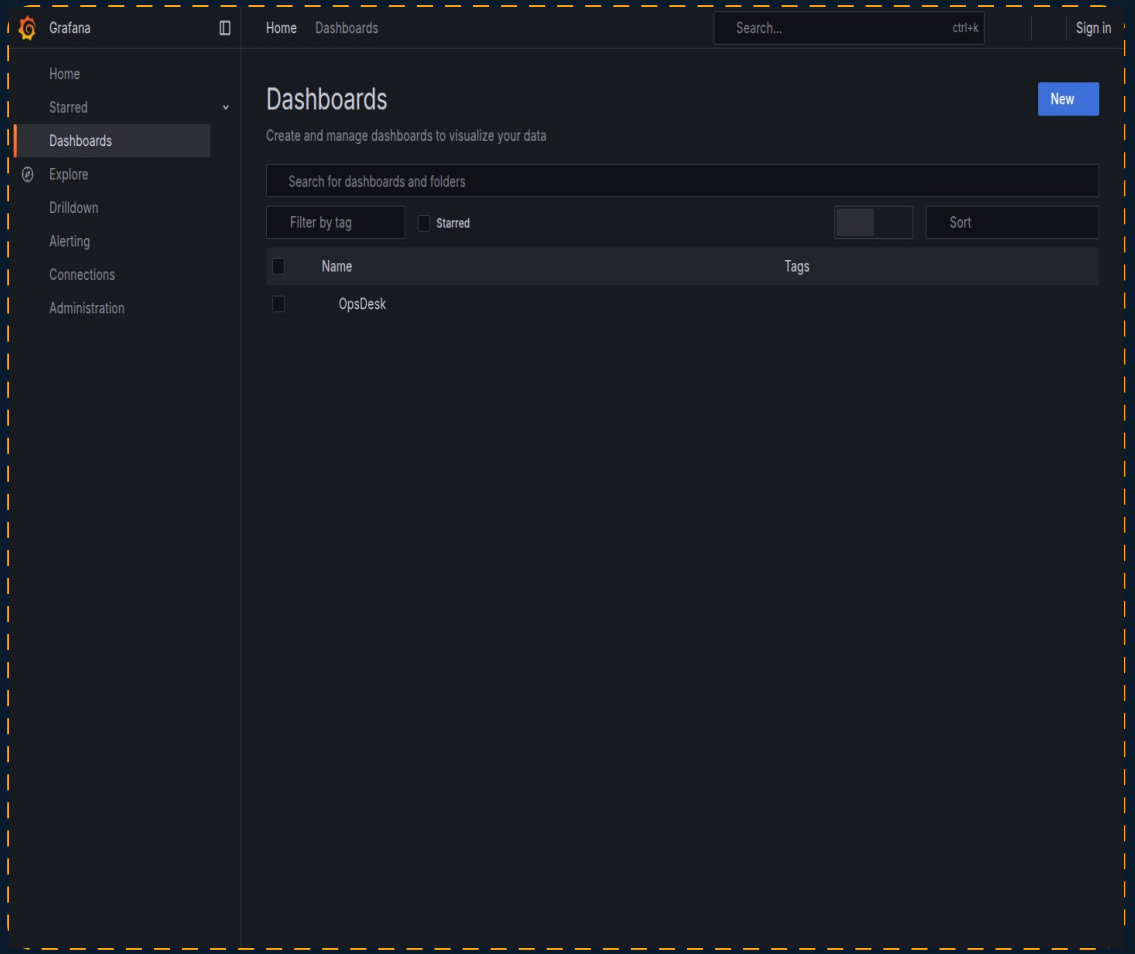
AWS Console — opsdesk-eks-prod status: Active

OBSERVABILITY

Observability & SRE Telemetry

Prometheus telemetry: Native /metrics endpoint exports HTTP request counters, latency histogram buckets, and CPU/RAM metrics.

Grafana SRE dashboard: Pre-configured panels track incident resolution rates, P95/P99 latency, and cluster saturation.



Grafana — OpsDesk SRE dashboard

KEY TAKEAWAYS

Complete 9-module DevOps capstone delivered on time, documented on GitHub, and verified on live cloud infrastructure.